

Examen de RFA, MUIARFID, DSIC, UPV, 21-10-2024

Apellidos, nombre:

Para cada cuestión, escoge una única opción de entre las dadas o no contestes.

Puntuación: $\max(0, (A - E/3) \cdot 0.25)$ con A aciertos y E errores; $A, E \in \{0, 1, \dots, 10\}$, $A + E \leq 10$.

1 ☐ Sea un problema de clasificación de objetos representados mediante vectores de $D = 2$ bits, $\mathbf{x} \in \{0, 1\}^D$, en $C = 2$ clases de probabilidades a priori $\pi_1 = 0.75$ y $\pi_2 = 0.25$. Se tiene un clasificador Naive Bayes Bernoulli que, como su nombre indica, asume que las funciones de probabilidad de las clases cumplen la asunción Naive Bayes,

$$p(\mathbf{x} \mid y = c, \boldsymbol{\mu}_c) = \prod_{d=1}^D \text{Ber}(x_d \mid \mu_{cd}) \quad \boldsymbol{\mu}_c = (\mu_{c1}, \dots, \mu_{cD})^t \in [0, 1]^D$$

donde μ_{cd} es la probabilidad de que $x_d = 1$ en la clase c . Sean $\boldsymbol{\mu}_1 = (0.7, 0.3)^t$ y $\boldsymbol{\mu}_2 = (0.2, 0.8)^t$. La probabilidad a posteriori P de que $\mathbf{x} = (0, 1)^t$ pertenezca a la clase 1 es:

1. $0.00 \leq P < 0.25$
2. $0.25 \leq P < 0.50$
3. $0.50 \leq P < 0.75$
4. $0.75 \leq P \leq 1.00$

Solución: La 2.

$$\begin{aligned} p(\mathbf{x} \mid y = 1, \boldsymbol{\mu}_1) &= \text{Ber}(x_1 \mid 0.7) \text{Ber}(x_2 \mid 0.3) = 0.3 \cdot 0.3 = 0.09 \\ p(\mathbf{x} \mid y = 2, \boldsymbol{\mu}_2) &= \text{Ber}(x_1 \mid 0.2) \text{Ber}(x_2 \mid 0.8) = 0.8 \cdot 0.8 = 0.64 \\ P = p(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}) &= \frac{\pi_1 p(\mathbf{x} \mid y = 1, \boldsymbol{\mu}_1)}{\pi_1 p(\mathbf{x} \mid y = 1, \boldsymbol{\mu}_1) + \pi_2 p(\mathbf{x} \mid y = 2, \boldsymbol{\mu}_2)} = \frac{0.75 \cdot 0.09}{0.75 \cdot 0.09 + 0.25 \cdot 0.64} = 0.2967 \end{aligned}$$

2 □ Sea un problema de clasificación de objetos representados mediante vectores de $D = 2$ variables reales en $C = 3$ clases de probabilidades a priori idénticas. Se tiene un clasificador Gaussiano de densidades condicionales con matrices de covarianzas común:

$$p(\mathbf{x} | y = c, \boldsymbol{\theta}) = \mathcal{N}_D(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}) \quad \text{donde} \quad \boldsymbol{\theta} = (\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_C, \text{vec}(\boldsymbol{\Sigma}))^t$$

con

$$\boldsymbol{\mu}_1 = (0, 0)^t, \quad \boldsymbol{\mu}_2 = (0, 4)^t, \quad \boldsymbol{\mu}_3 = (4, 4)^t \quad \text{y} \quad \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

La región de decisión de la clase 2 es:

1. $\mathcal{R}_2 = \{\mathbf{x} : x_1 > 2, x_2 > 2\}$
2. $\mathcal{R}_2 = \{\mathbf{x} : x_1 > 2, x_2 < 2\}$
3. $\mathcal{R}_2 = \{\mathbf{x} : x_1 < 2, x_2 > 2\}$
4. $\mathcal{R}_2 = \{\mathbf{x} : x_1 < 2, x_2 < 2\}$

Solución: La 3. Discriminantes lineales simplificadas (ignorando términos aditivos independientes de c):

$$\log p(y = c | \mathbf{x}, \boldsymbol{\theta}) \propto g_c(\mathbf{x}) = \boldsymbol{\beta}_c^t \mathbf{x} + \gamma_c \quad \text{con} \quad \boldsymbol{\beta}_c = \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c = \boldsymbol{\mu}_c \quad \text{y} \quad \gamma_c = -\frac{1}{2} \boldsymbol{\mu}_c^t \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c = -\frac{1}{2} \boldsymbol{\mu}_c^t \boldsymbol{\mu}_c$$

$$g_1(\mathbf{x}) = \boldsymbol{\mu}_1^t \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_1^t \boldsymbol{\mu}_1 = 0$$

$$g_2(\mathbf{x}) = \boldsymbol{\mu}_2^t \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_2^t \boldsymbol{\mu}_2 = 4x_2 - 8$$

$$g_3(\mathbf{x}) = \boldsymbol{\mu}_3^t \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_3^t \boldsymbol{\mu}_3 = 4x_1 + 4x_2 - 16$$

$$\begin{aligned} \mathcal{R}_2 &= \{\mathbf{x} : g_2(\mathbf{x}) > g_1(\mathbf{x}), g_2(\mathbf{x}) > g_3(\mathbf{x})\} \\ &= \{\mathbf{x} : 4x_2 - 8 > 0, 4x_2 - 8 > 4x_1 + 4x_2 - 16\} \\ &= \{\mathbf{x} : x_1 < 2, x_2 > 2\} \end{aligned}$$

3 ☐ Sea un problema de regresión de objetos representados mediante vectores de $D = 1$ reales, $\mathbf{x} \in \mathbb{R}^D$. Se ha ajustado un modelo de regresión lineal con datos en notación homogénea (sesgo absorbido en $\mathbf{w}$),

$$p(y | \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y | \mu, \sigma^2) \quad \text{con} \quad \mu = \mathbf{w}^t \mathbf{x} \quad \text{y} \quad \mathbf{x} = (1, x_1)^t \in \mathbb{R}^2$$

a partir de $N = 4$ datos, $\mathbf{X} = [1, 0; 1, 1; 1, 1; 1, 3]$ y $\mathbf{y} = (3, 1.5, 2.5, 0)^t$. Los parámetros ajustados son:

$$\hat{\mathbf{w}} = \begin{pmatrix} 3 \\ -1 \end{pmatrix} \quad \text{y} \quad \hat{\sigma}^2 = 0.125$$

El coeficiente de determinación R^2 del modelo con los datos empleados en su ajuste es:

1. $R^2 < 0.7$
2. $0.7 \leq R^2 < 0.8$
3. $0.8 \leq R^2 < 0.9$
4. $R^2 \geq 0.9$

Solución: La 4.

$$\text{RSS}(\hat{\mathbf{w}}) = N \hat{\sigma}^2 = 0.5$$

$$\bar{y} = 7/4 = 1.75$$

$$\begin{aligned} \text{TSS}(\hat{\mathbf{w}}) &= (3 - 1.75)^2 + (1.5 - 1.75)^2 + (2.5 - 1.75)^2 + (0 - 1.75)^2 \\ &= 1.5625 + 0.0625 + 0.5625 + 3.0625 = 5.25 \end{aligned}$$

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = 1 - 0.5/5.25 = 0.9048$$

```
In [ ]: import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
X = np.array([[1,0],[1,1],[1,1],[1,3]]); y = np.array([3,1.5,2.5,0]); N = len(y)
w = np.linalg.inv(X.T @ X) @ X.T @ y; s = 1 / N * np.square(y - X @ w).sum(); print(w, s);
RSS = N * s; TSS = np.square(y - y.mean()).sum(); R2 = 1 - RSS / TSS; print(f'{R2:.4f}')
```

[3.00 -1.00] 0.125
0.9048

```
In [ ]: from sklearn.linear_model import LinearRegression; from sklearn.metrics import r2_score as R2
lr = LinearRegression(fit_intercept=False).fit(X, y); y_pred = lr.predict(X); print(f'{R2(y, y_pred):.4f}')
```

0.9048

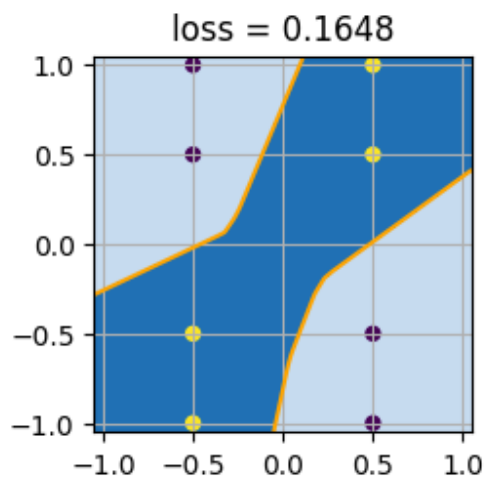
```
In [ ]: import numpy as np; np.set_printoptions(precision=3); import matplotlib.pyplot as plt
import pandas as pd; pd.set_option('display.max_rows', 4)
import silence_tensorflow.auto; import keras; keras.utils.set_random_seed(23)
```

```
In [ ]: ds = pd.read_csv('04_MLP_clasif.csv').to_numpy()
X = ds[:, :2]; y = ds[:, 2] #y = keras.utils.to_categorical(ds[:, 2], 2)
X = X / np.max(np.abs(X))
print(np.c_[X, y])
```

```
[[-0.5  1.  0. ]
 [-0.5  0.5 0. ]
 [ 0.5 -0.5 0. ]
 [ 0.5 -1.  0. ]
 [ 0.5  1.  1. ]
 [ 0.5  0.5 1. ]
 [-0.5 -0.5 1. ]
 [-0.5 -1.  1. ]]
```

```
In [ ]: M = keras.Sequential([
    keras.layers.Input((2,)),
    keras.layers.Dense(6, activation=keras.activations.relu),
    keras.layers.Dense(2, activation=keras.activations.relu),
    keras.layers.Dense(1, activation=keras.activations.sigmoid)])
opt = keras.optimizers.Adam(learning_rate=0.01)
M.compile(loss='binary_crossentropy', optimizer=opt, metrics=['accuracy'])
M.fit(X, y, epochs=50, verbose=0);
```

```
In [ ]: fig, ax = plt.subplots(1, 1, figsize=(2.5, 2.5))
ax.grid(); ticks = [-1., -.5, .0, .5, 1.]; ax.set_xticks(ticks); ax.set_yticks(ticks)
ax.set_title(f'loss = {M.evaluate(X, y, verbose=0)[0]:.4f}')
R = np.linspace(-1.05, 1.05, 50); x1, x2 = np.meshgrid(R, R)
XX = np.c_[np.ravel(x1), np.ravel(x2)]; yy = M.predict(XX, verbose=0)
ax.contour(x1, x2, yy.reshape(x1.shape), 1, colors='orange', linestyle='solid')
ax.contourf(x1, x2, yy.reshape(x1.shape), 1, cmap='Blues')
ax.scatter(*X.T, c=y, s=25);
```



```
In [ ]: M.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6)	18
dense_1 (Dense)	(None, 2)	14
dense_2 (Dense)	(None, 1)	3

Total params: 107 (432.00 B)
Trainable params: 35 (140.00 B)
Non-trainable params: 0 (0.00 B)
Optimizer params: 72 (292.00 B)

```
In [ ]: W1t, b1 = M.layers[0].get_weights(); print(f"W1t={W1t}\n b1={b1}")
W2t, b2 = M.layers[1].get_weights(); print(f"W2t={W2t}\n b2={b2}")
W3t, b3 = M.layers[2].get_weights(); print(f"W3t={W3t}\n b3={b3}")
```

W1t=[[-0.357 1.275 -0.203 -0.328 -0.998 0.771]
 [-0.815 -1.113 -0.024 -0.495 0.674 0.281]]
b1=[-0.062 -0.08 0.448 -0.08 -0.108 0.152]
W2t=[[0.611 -0.43]
 [-0.745 0.934]
 [0.292 0.212]
 [0.387 -0.312]
 [-0.807 1.098]
 [0.954 0.422]]
b2=[0.431 0.24]
W3t=[[1.961]
 [-1.291]]
b3=[0.092]

4 □ Sea el MLP para datos 2d de dos clases, $\mathbf{x} \in [-1, 1]^2$, $y \in \{0, 1\}$:

$$\begin{aligned} \mathbf{z}^{(1)} &= \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \in \mathbb{R}^6 & \mathbf{x}^{(2)} &= \text{ReLU}(\mathbf{z}^{(1)}) \in \mathbb{R}^6 \\ \mathbf{z}^{(2)} &= \mathbf{W}^{(2)} \mathbf{x}^{(2)} + \mathbf{b}^{(2)} \in \mathbb{R}^2 & \mathbf{x}^{(3)} &= \text{ReLU}(\mathbf{z}^{(2)}) \in \mathbb{R}^2 \\ \mathbf{z}^{(3)} &= \mathbf{W}^{(3)} \mathbf{x}^{(3)} + \mathbf{b}^{(3)} \in \mathbb{R} & \hat{y} &= \sigma(\mathbf{z}^{(3)}) \in \mathbb{R} \end{aligned}$$

donde:

```
In [ ]: from sympy import Matrix, latex; from IPython.display import Latex
W1t_latex = rf"$$$\\mathbf{\\{W\\}}^{{{\\{1\\}}t}}=\\{\\text{latex}(\\text{Matrix}(W1t).\\text{applyfunc}(\\text{lambda x: round(x, 3)}))\\}$$$"
b1t_latex = rf"$$$\\mathbf{\\{b\\}}^{{{\\{1\\}}t}}=\\{\\text{latex}(\\text{Matrix}(b1).T.\\text{applyfunc}(\\text{lambda x: round(x, 3)}))\\}$$$"
W2_latex = rf"$$$\\mathbf{\\{W\\}}^{{{\\{2\\}}}}=\\{\\text{latex}(\\text{Matrix}(W2t).T.\\text{applyfunc}(\\text{lambda x: round(x, 3)}))\\}$$$"
b2t_latex = rf"$$$\\mathbf{\\{b\\}}^{{{\\{2\\}}t}}=\\{\\text{latex}(\\text{Matrix}(b2).T.\\text{applyfunc}(\\text{lambda x: round(x, 3)}))\\}$$$"
W3_latex = rf"$$$\\mathbf{\\{W\\}}^{{{\\{3\\}}}}=\\{\\text{latex}(\\text{Matrix}(W3t).T.\\text{applyfunc}(\\text{lambda x: round(x, 3)}))\\}$$$"
b3_latex = rf"$$$\\mathbf{\\{b\\}}^{{{\\{3\\}}}}=\\{\\text{latex}(\\text{Matrix}(b3).\\text{applyfunc}(\\text{lambda x: round(x, 3)}))\\}$$$"
Latex(W1t_latex+"\n"+b1t_latex+"\n"+W2_latex+"\n"+b2t_latex+"\n"+W3_latex+"\n"+b3_latex)
```

Out[]:

$$\begin{aligned} \mathbf{W}^{(1)t} &= \begin{bmatrix} -0.357 & 1.275 & -0.203 & -0.328 & -0.998 & 0.771 \\ -0.815 & -1.113 & -0.024 & -0.495 & 0.674 & 0.281 \end{bmatrix} \\ \mathbf{b}^{(1)t} &= [-0.062 \quad -0.08 \quad 0.448 \quad -0.08 \quad -0.108 \quad 0.152] \\ \mathbf{W}^{(2)} &= \begin{bmatrix} 0.611 & -0.745 & 0.292 & 0.387 & -0.807 & 0.954 \\ -0.43 & 0.934 & 0.212 & -0.312 & 1.098 & 0.422 \end{bmatrix} \\ \mathbf{b}^{(2)t} &= [0.431 \quad 0.24] \\ \mathbf{W}^{(3)} &= [1.961 \quad -1.291] \\ \mathbf{b}^{(3)} &= [0.092] \end{aligned}$$

Haciendo uso del MLP dado y la regla de decisión MAP, las clases de $\mathbf{x}_1 = (0, 0)^t$ y $\mathbf{x}_2 = (1, 0)^t$ son, respectivamente:

1. $y_1 = 0$, $y_2 = 0$
2. $y_1 = 0$, $y_2 = 1$
3. $y_1 = 1$, $y_2 = 0$
4. $y_1 = 1$, $y_2 = 1$

Solución: La 3. El MLP dado estima la probabilidad $p(y = 1 \mid \mathbf{x})$:

```
In [ ]: import numpy as np; from scipy.special import expit
X = np.array([[0, 0], [1, 0]])
for n in range(X.shape[0]):
    z1 = X[n, :] @ W1t + b1; x2 = np.maximum(z1, 0.0)
    z2 = x2 @ W2t + b2; x3 = np.maximum(z2, 0.0)
    z3 = x3 @ W3t + b3; hy = expit(z3)
    print(hy)
M.predict(X, verbose=0)

[0.724]
[0.22]
array([[0.724],
       [0.22 ]], dtype=float32)

Out[ ]:
```

5 □ Sea el MLP $\mathbf{x} \in \{0, 1\}^2$, $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}_1$, $\mathbf{h} = \text{ReLU}(\mathbf{z})$, $\hat{y} = \mathbf{V}\mathbf{h} + b_2$, con θ :

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \mathbf{b}_1 = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \quad \mathbf{V} = [1 \quad -1] \quad b_2 = 0$$

Se está aplicando backprop para calcular el gradiente del error cuadrático de θ respecto a $(\mathbf{x}, y)$, $\mathbf{x} = (1, 1)^t$, $y = -1$. Se ha completado el forward:

$$\mathbf{z} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad \mathbf{h} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad \hat{y} = 1 \quad \mathcal{L} = 4$$

En backward, la Jacobiana del error cuadrático respecto a $\mathbf{h}$ es:

$$\mathbf{J}(\mathbf{h}) = (a, b)$$

tal que:

1. $a > 0$, y $b > 0$
2. $a > 0$ y $b \leq 0$
3. $a \leq 0$ y $b > 0$
4. $a \leq 0$ y $b \leq 0$

Solución: La 2.

$$\mathbf{J}(\mathbf{h}) = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{h}} = 2(\hat{y} - y) \mathbf{V} = 2(1 - (-1)) [1 \quad -1] = [4 \quad -4]$$

6 □ Sea el MLP $\mathbf{x} \in \{0, 1\}^2$, $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}_1$, $\mathbf{h} = \text{ReLU}(\mathbf{z})$, $\mathbf{a} = \mathbf{V}\mathbf{h} + \mathbf{b}_2$, $\hat{\mathbf{y}} = \mathcal{S}(\mathbf{a})$, con θ :

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \mathbf{b}_1 = \begin{bmatrix} -1 \\ 0.5 \end{bmatrix} \quad \mathbf{V} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad \mathbf{b}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Se está aplicando backprop para calcular el gradiente de la entropía cruzada de θ respecto a $(\mathbf{x}, \mathbf{y})$, $\mathbf{x} = (1, 1)^t$, $\mathbf{y} = (1, 0)^t$. Se ha completado el forward:

$$\mathbf{z} = \begin{pmatrix} 1 \\ 2.5 \end{pmatrix} \quad \mathbf{h} = \begin{pmatrix} 1 \\ 2.5 \end{pmatrix} \quad \mathbf{a} = \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} \quad \hat{\mathbf{y}} = \begin{pmatrix} 0.2689 \\ 0.7311 \end{pmatrix} \quad \mathcal{L} = 1.3133$$

La Jacobiana de la entropía cruzada respecto a $\mathbf{W}$ es:

$$\mathbf{J}(\mathbf{W}) = \begin{pmatrix} a & b \\ a & b \end{pmatrix}$$

tal que:

1. $a > 0$, y $b > 0$
2. $a > 0$ y $b \leq 0$
3. $a \leq 0$ y $b > 0$
4. $a \leq 0$ y $b \leq 0$

Solución: La 3.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{a}} &= (\hat{\mathbf{y}} - \mathbf{y})^t = \left(\begin{pmatrix} 0.2689 \\ 0.7311 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix} \right)^t = (-0.7311, 0.7311) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{h}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{h}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}} \mathbf{V} = (-0.7311, 0.7311) \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} = (-1.4622, 1.4622) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{z}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}} \odot H(\mathbf{z})^t = (-1.4622, 1.4622)(1, 1) = (-1.4622, 1.4622) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \mathbf{x} \frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} (-1.4622, 1.4622) = \begin{pmatrix} -1.4622 & 1.4622 \\ -1.4622 & 1.4622 \end{pmatrix} \end{aligned}$$

7 ☐ Al entrenar modelos muy profundos, el gradiente suele hacerse muy pequeño (desaparece) o muy grande (explota). La solución estándar para los gradientes que explotan consiste en recortarlos de acuerdo con una norma máxima y valores absolutos máximos (gradient clipping). Para los que desaparecen se han propuesto diversas soluciones. En relación con las mismas, indica la afirmación incorrecta o escoge la última opción si las tres primeras son correctas:

1. Modificar la arquitectura para que las actualizaciones sean aditivas en lugar de multiplicativas.
2. Modificar la arquitectura para estandarizar las activaciones de cada capa, de manera que la distribución de las activaciones sobre los datos se mantenga constante durante el entrenamiento.
3. Escoger los valores iniciales de los parámetros cuidadosamente.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

8 □ Sean la entrada 5×5 y filtro 3×3 :

$$\mathbf{X} = \begin{bmatrix} 3 & 3 & 2 & 1 & 0 \\ 0 & 0 & 1 & 3 & 1 \\ 3 & 1 & 2 & 2 & 3 \\ 2 & 0 & 0 & 2 & 2 \\ 2 & 0 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 0 & 1 & 2 \\ 2 & 2 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$

La convolución 2d de $\mathbf{X}$ con $\mathbf{W}$, $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X}$, con padding $p_h = p_w = 1$ y strides $s_h = s_w = 2$ es

$$\mathbf{Y} = \begin{bmatrix} y_{00} & y_{01} & \cdots & y_{0,m-1} \\ y_{10} & y_{11} & \cdots & y_{1,m-1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{n-1,0} & y_{n-1,1} & \cdots & y_{n-1,m-1} \end{bmatrix}$$

tal que:

1. $n = m = 3$ y $y_{11} < 15$
2. $n = m = 3$ y $y_{11} \geq 15$
3. $n = m = 4$ y $y_{11} < 15$
4. $n = m = 4$ y $y_{11} \geq 15$

Solución: La 2. Tamaño de la salida con $x_h = x_w = 5$, $f_h = f_w = 3$, $p_h = p_w = 1$, $s_h = s_w = 2$:

$$\left\lfloor \frac{x_h + 2p_h - f_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{x_w + 2p_w - f_w + s_w}{s_w} \right\rfloor = \left\lfloor \frac{5 + 2 - 3 + 2}{2} \right\rfloor \times \left\lfloor \frac{5 + 2 - 3 + 2}{2} \right\rfloor = 3 \times 3$$

$$\mathbf{Y} = \begin{bmatrix} y_{00} & y_{01} & y_{02} \\ y_{10} & y_{11} & y_{12} \\ y_{20} & y_{21} & y_{22} \end{bmatrix} \quad \text{con} \quad y_{ij} = \sum_{u=-1}^1 \sum_{v=-1}^1 w_{uv} x_{si+u, sj+v}$$

$$y_{00} = w_{-1-1}x_{-1-1} + w_{-10}x_{-10} + w_{-11}x_{-11} + w_{0-1}x_{0-1} + w_{00}x_{00} + w_{01}x_{01} + w_{1-1}x_{1-1} + w_{10}x_{10} + w_{11}x_{11} = 6$$

$$y_{01} = w_{-1-1}x_{-11} + w_{-10}x_{-12} + w_{-11}x_{-13} + w_{0-1}x_{01} + w_{00}x_{02} + w_{01}x_{03} + w_{1-1}x_{11} + w_{10}x_{12} + w_{11}x_{13} = 17$$

$$y_{02} = w_{-1-1}x_{-13} + w_{-10}x_{-14} + w_{-11}x_{-15} + w_{0-1}x_{03} + w_{00}x_{04} + w_{01}x_{05} + w_{1-1}x_{13} + w_{10}x_{14} + w_{11}x_{15} = 3$$

$$y_{10} = w_{-1-1}x_{1-1} + w_{-10}x_{10} + w_{-11}x_{11} + w_{0-1}x_{2-1} + w_{00}x_{20} + w_{01}x_{21} + w_{1-1}x_{3-1} + w_{10}x_{30} + w_{11}x_{31} = 8$$

$$y_{11} = w_{-1-1}x_{11} + w_{-10}x_{12} + w_{-11}x_{13} + w_{0-1}x_{21} + w_{00}x_{22} + w_{01}x_{23} + w_{1-1}x_{31} + w_{10}x_{32} + w_{11}x_{33} = 17$$

$$y_{12} = w_{-1-1}x_{13} + w_{-10}x_{14} + w_{-11}x_{15} + w_{0-1}x_{23} + w_{00}x_{24} + w_{01}x_{25} + w_{1-1}x_{33} + w_{10}x_{34} + w_{11}x_{35} = 13$$

$$y_{20} = w_{-1-1}x_{3-1} + w_{-10}x_{30} + w_{-11}x_{31} + w_{0-1}x_{4-1} + w_{00}x_{40} + w_{01}x_{41} + w_{1-1}x_{5-1} + w_{10}x_{50} + w_{11}x_{51} = 6$$

$$y_{21} = w_{-1-1}x_{31} + w_{-10}x_{32} + w_{-11}x_{33} + w_{0-1}x_{41} + w_{00}x_{42} + w_{01}x_{43} + w_{1-1}x_{51} + w_{10}x_{52} + w_{11}x_{53} = 4$$

$$y_{22} = w_{-1-1}x_{33} + w_{-10}x_{34} + w_{-11}x_{35} + w_{0-1}x_{43} + w_{00}x_{44} + w_{01}x_{45} + w_{1-1}x_{53} + w_{10}x_{54} + w_{11}x_{55} = 4$$

0 ₀	0 ₁	0 ₂	0	0	0	0
0 ₂	3 ₂	3 ₀	2	1	0	0
0 ₀	0 ₁	0 ₂	1	3	1	0
0	3	1	2	2	3	0
0	2	0	0	2	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

0	0	0 ₀	0 ₁	0 ₂	0	0
0	3	3 ₂	2 ₂	1 ₀	0	0
0	0	0 ₀	1 ₁	3 ₂	1	0
0	3	1	2	2	3	0
0	2	0	0	2	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

0	0	0	0	0 ₀	0 ₁	0 ₂
0	3	3	2	1 ₂	0 ₂	0 ₀
0	0	0	1	3 ₀	1 ₁	0 ₂
0	3	1	2	2	3	0
0	2	0	0	2	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0 ₀	0 ₁	0 ₂	1	3	1	0
0 ₂	3 ₂	1 ₀	2	2	3	0
0 ₀	2 ₁	0 ₂	0	2	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0	0	0 ₀	1 ₁	3 ₂	1	0
0	3	1 ₂	2 ₂	2 ₀	3	0
0	2	0 ₀	0 ₁	2 ₂	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0	0	0	1	3 ₀	1 ₁	0 ₂
0	3	1	2	2 ₂	3 ₂	0 ₀
0	2	0	0	2 ₀	2 ₁	0 ₂
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0	0	0	1	3	1	0
0	3	1	2	2	3	0
0 ₀	2 ₁	0 ₂	0	2	2	0
0 ₂	2 ₂	0 ₀	0	0	1	0
0 ₀	0 ₁	0 ₂	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0	0	0	1	3	1	0
0	3	1	2	2	3	0
0	2	0 ₀	0 ₁	2 ₂	2	0
0	2	0 ₂	0 ₂	0 ₀	1	0
0	0	0 ₀	0 ₁	0 ₂	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0	0	0	1	3	1	0
0	3	1	2	2	3	0
0	2	0	0	2 ₀	2 ₁	0 ₂
0	2	0	0	0 ₂	1 ₂	0 ₀
0	0	0	0	0 ₀	0 ₁	0 ₂

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

9 ☐ El clasificador por los K-vecinos más cercanos (KNN) es un clasificador no paramétrico con el que se obtiene una estimación directa de la probabilidad a posteriori $p(y = c \mid \mathbf{x})$. Para ello, dada una entrada $\mathbf{x}$, KNN busca K ejemplos más cercanos de $\mathbf{x}$ en el conjunto de entrenamiento, $N_K(\mathbf{x}, \mathcal{D})$, con lo que $p(y = c \mid \mathbf{x})$ quedaría definida como:

1. $p(y = c \mid \mathbf{x}) = \frac{1}{|\mathcal{D}|} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n \neq c)$
2. $p(y = c \mid \mathbf{x}) = \frac{1}{K} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n = c)$
3. $p(y = c \mid \mathbf{x}) = \frac{1}{K} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n \neq c)$
4. $p(y = c \mid \mathbf{x}) = \frac{1}{|\mathcal{D}|} \sum_{n \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_n = c)$

Solución: La 2.

10 ☐ Indica cuál de las siguientes funciones es kernel densidad (o escoge la última si ninguna lo es):

1. $\mathcal{K}(x) = \mathbb{I}(|x| \leq 2)$

2. $\mathcal{K}(x) = \frac{1}{2}\mathbb{I}(|x| \leq 2)$

3. $\mathcal{K}(x) = \frac{1}{4}\mathbb{I}(|x| \leq 2)$

4. Ninguna de las funciones anteriores es kernel densidad

Solución: La 3.